
PROYECTO 2 – SISTEMA DE CATÁLOGO PARA UNA CADENA DE LIBRERÍAS.

202500708 – David Antonio Meza Silva

Resumen

El presente ensayo expone el diseño e implementación de un sistema de catálogo bibliográfico que organiza libros de forma jerárquica en categorías y subcategorías, se alimenta de archivos XML y se gestiona desde una interfaz web. El proyecto responde a la necesidad, vigente en sistemas de información con catálogos en constante crecimiento, de mantener búsquedas eficientes cuando el volumen supera los cien mil libros, sin recurrir a colecciones nativas del lenguaje C#.

Se construyeron desde cero árboles AVL para indexar los libros por su ISBN y las categorías por nombre, junto con reportes gráficos generados con Graphviz y una carga incremental de XML tolerante a errores. A nivel técnico, los árboles AVL permiten mantener costos logarítmicos de búsqueda, inserción y eliminación, evitando la degradación que sufriría un árbol binario simple ante datos ordenados.

Se concluye que el balanceo automático y los índices redundantes permiten un catálogo eficiente, ordenado y consistente, además de reforzar, en el ámbito formativo, la comprensión de las estructuras de datos y de la programación orientada a objetos.

Palabras Clave:

Árboles AVL; estructuras de datos; catálogo bibliográfico; interfaz web; reportes en Graphviz.

Abstract

This essay presents the design and implementation of a bibliographic catalog system that organizes books hierarchically into categories and subcategories, is populated from XML files, and is managed through a web interface. The project addresses an ongoing need in information systems with constantly growing catalogs: maintaining efficient searches when the volume exceeds one hundred thousand books, without relying on built-in C# collections.

AVL trees were implemented from scratch to index books by ISBN and categories by name, together with graphical reports generated with Graphviz and an error-tolerant incremental XML loader. At a technical level, AVL trees keep search, insertion, and deletion costs logarithmic, preventing the degradation that a plain binary search tree would suffer with sorted input.

The work concludes that automatic balancing and redundant indexes enable an efficient, ordered, and consistent catalog, while also reinforcing, in an educational context, the understanding of data structures and object-oriented programming.

Keywords:

AVL trees; data structures; bibliographic catalog; web interface; Graphviz reports.

Introducción

El manejo eficiente de grandes volúmenes de información es uno de los desafíos principales en el desarrollo de software moderno. El presente proyecto aborda la problemática de una cadena de librerías que requiere administrar un catálogo de más de cien mil libros distribuidos en relaciones jerárquicas de categorías y subcategorías (Facultad de Ingeniería, 2026).

El sistema anterior presentaba deficiencias críticas en los tiempos de respuesta debido a búsquedas secuenciales en listas extensas. Para solucionar esto, se desarrolló un sistema enfocado en la localización inmediata de información, aplicando Tipos de Datos Abstractos (TDAs) no lineales y gestión directa de memoria dinámica mediante el lenguaje C#.

Este ensayo expone las decisiones de diseño detrás de la solución y busca responder una pregunta central: ¿cómo mantener búsquedas eficientes y una organización jerárquica en un catálogo de esta escala sin recurrir a colecciones nativas del lenguaje?.

Desarrollo del tema

a. Abstracción del Problema y Programación Orientada a Objetos

Para modelar la solución, se analizó primero la estructura de una biblioteca física, la cual organiza su contenido de lo general a lo específico (áreas, categorías, subcategorías y, finalmente, libros). Utilizando el paradigma de programación orientada a objetos, se abstraigo esta realidad en cuatro entidades fundamentales (Anexos A y B):

- **NodoLibro:** Encapsula el ISBN, el título, el autor y una referencia a la categoría en que está clasificado.
- **NodoCategoría:** Contenedor jerárquico con un nombre, una referencia a su

categoría padre, sus subcategorías y los libros clasificados directamente en ella.

- **ArbolCategorias:** Administra la jerarquía completa y un índice global de categorías por nombre.
- **Catalogo:** Controlador central que coordina el registro, la búsqueda y la eliminación de libros y mantiene la coherencia entre los índices.

b. Tipos de Datos Abstractos (TDAs) propios

La restricción de no utilizar colecciones nativas de C# obligó a construir desde cero cada estructura de almacenamiento, enlazando nodos mediante referencias. La decisión más importante fue la estructura que indexaría los libros. Una lista enlazada obliga a recorrer el catálogo completo en cada búsqueda, que es justamente el problema del sistema anterior. Un árbol binario de búsqueda mejora el caso promedio, pero su forma depende del orden de ingreso: si los ISBN llegan en orden ascendente, cada inserción se agrega a la derecha y el árbol se degenera en una lista de altura n .

Con 100,000 libros esto implica hasta 100,000 comparaciones por búsqueda y una recursión de esa misma profundidad.

Por ello se implementó un árbol AVL, un árbol binario de búsqueda que se autobalancea. Cada nodo almacena su altura y, tras cada inserción o eliminación, se calcula su factor de balance, es decir, la altura del subárbol izquierdo menos la del derecho. Cuando el valor sale del rango de -1 a 1 , el equilibrio se restablece con una de cuatro rotaciones (izquierda-izquierda, derecha-derecha, izquierda-derecha y derecha-izquierda;

ver Anexo D). Esto acota la altura a cerca de $1.44 \cdot \log_2(n)$: con 100,000 libros, unos 23 niveles como máximo. La misma estructura, con el nombre como llave, indexa las categorías. Al usar el ISBN como llave, el recorrido in-order entrega los libros en orden ascendente sin ordenamiento adicional, y el menor y el mayor se obtienen descendiendo por la rama izquierda y la derecha, respectivamente.

c. **Organización de libros y Categorías**

Cada `NodoCategoria` contiene un AVL de subcategorías y un AVL con los libros clasificados directamente en ella, lo que permite representar jerarquías de profundidad arbitraria y recorrer cada nivel en orden alfabético, como exige el enunciado. La comparación de nombres ignora mayúsculas y acentos, de modo que «Matematicas» encuentra «Matemáticas»; esto facilita la consulta y evita que dos nombres que solo difieren en esos aspectos se registren como categorías distintas.

Buscar una subcategoría profunda recorriendo la jerarquía desde la raíz sería costoso. Por eso `ArbolCategorias` mantiene, además de las categorías principales, un índice global: un AVL con todas las categorías por nombre. Con él, cualquier categoría se localiza en $O(\log c)$ sin importar su nivel, y se impone la unicidad de nombres en toda la jerarquía. Los libros siguen la misma lógica: existe un AVL global por ISBN y, en cada categoría, un AVL con sus libros. Ambos apuntan al mismo objeto `NodoLibro` (Figura 1), por lo que los datos no se duplican; el costo es una referencia adicional por libro, a cambio de acceso directo por ISBN y por categoría. Como el libro conoce su categoría, eliminarlo consiste

en quitarlo de ambos árboles en tiempo logarítmico. Para obtener los libros de una categoría y de todas sus subcategorías, el sistema recorre el subárbol e inserta cada libro en un AVL temporal, que los entrega ordenados por ISBN. Los recorridos se realizan mediante delegados (`AccionLibro` y `AccionCategoria`) que reciben cada elemento en orden: la estructura decide cómo recorrerse y quien la usa decide qué hacer con cada nodo (contar, generar reportes o construir respuestas), sin exponer colecciones intermedias. La Tabla I resume los costos, con n libros, c categorías y m libros del subárbol consultado.

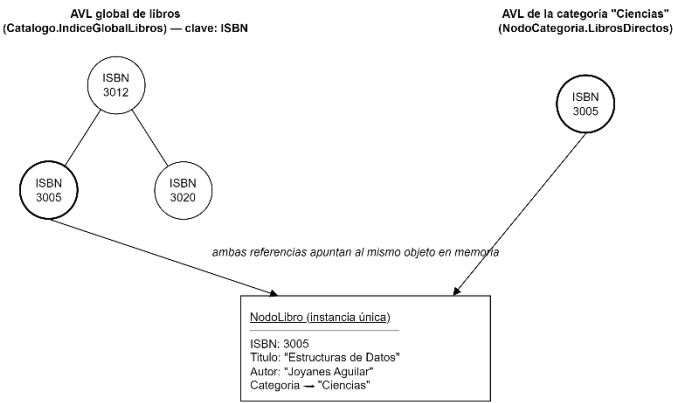


Figura 1. Un mismo `NodoLibro` referenciado desde el AVL global de libros y desde el AVL de su categoría. Fuente: Elaboración propia

Tabla I. Costo de las operaciones principales.

Operación	Costo
Buscar por ISBN	$O(\log n)$
Registrar Libro	$O(\log c + \log n)$
Eliminar Libro	$O(\log n)$
Libro con mayor o menor ISBN	$O(\log n)$
Listar Libros Ascendentemente	$O(n)$

Buscar Categoría por nombre	$O(\log c)$
Libros de una categoría y sus subcategorías	$O(m \log m)$

d. Carga Incremental de Archivos XML

El archivo de entrada declara categorías, con un atributo opcional que indica su padre, y libros; ambas listas son opcionales. La carga es incremental: no reinicia el catálogo, por lo que pueden procesarse varios archivos sucesivos, y la reinicialización es una acción explícita del usuario. Los archivos originales nunca se modifican.

Dado que el orden de declaración no está garantizado, una categoría puede aparecer antes que su padre. Para resolverlo, el cargador recorre la lista de categorías en pasadas sucesivas: en cada una agrega las que son raíz o cuyo padre ya existe, y repite mientras se registre algún progreso. Al terminar, las categorías cuyo padre nunca fue declarado se reportan como avisos. El número de pasadas está acotado por la profundidad de la jerarquía (Anexo C). Antes de estas pasadas, una revisión previa detecta categorías repetidas o ya existentes; para ello aprovecha un AVL temporal cuya inserción rechaza nombres duplicados.

Los libros se validan uno a uno: campos completos, ISBN entero, categoría existente e ISBN no repetido. Cada elemento se procesa de forma independiente, de modo que un registro inválido genera un aviso, pero no interrumpe la carga de los demás. Al finalizar, el sistema informa cuántas categorías y libros se agregaron, junto con la lista de avisos.

e. Reportes con Graphviz e Interfaz Web

Los reportes se generan en dos pasos: el sistema construye un archivo de texto en lenguaje DOT que describe el grafo y luego invoca la herramienta Graphviz (Gansner & North, 2000) como proceso externo para producir la imagen PNG. Existen dos reportes: la gráfica de los libros de una categoría y sus subcategorías, dibujados como una cadena en orden ascendente de ISBN, y la jerarquía del catálogo, completa o desde una subcategoría elegida, que representa las categorías como carpetas y los libros como registros. Los caracteres especiales de títulos y nombres (comillas, llaves, barras verticales y signos de menor y mayor) se escapan antes de escribir el DOT, para que datos arbitrarios del XML no invaliden el grafo.

La interfaz es una página web estática que consume una API REST implementada con ASP.NET Core. Un único objeto Catalogo se mantiene en memoria y cada operación (cargar XML, registrar, buscar, eliminar, consultar extremos, listar y graficar) corresponde a un endpoint. Los datos provenientes de los archivos se insertan en la página como texto o con codificación HTML, evitando que un título con etiquetas altere la interfaz.

Conclusiones

El desarrollo del sistema confirma que es posible administrar un catálogo de gran escala sin colecciones nativas del lenguaje, apoyándose en tipos de dato abstracto propios. La elección del árbol AVL fue determinante: mientras un árbol binario de búsqueda simple depende del orden de ingreso, el AVL garantiza costos logarítmicos en búsqueda, inserción y eliminación, requisito central del enunciado.

Los índices redundantes (global y por categoría) reflejan un compromiso conocido entre memoria y tiempo: una referencia adicional por elemento permite acceder a un libro por ISBN y a una categoría por nombre sin recorrer toda la estructura. La carga incremental, con pasadas sucesivas y avisos por elemento, permite que el sistema tolere archivos desordenados o con errores sin perder los datos válidos.

Entre las limitaciones están que los datos residen únicamente en memoria y se pierden al cerrar el sistema, que se requiere Graphviz instalado y que las gráficas de categorías con miles de libros pueden resultar poco legibles. Como trabajo futuro se propone un índice secundario por título, dado que el planteamiento del problema menciona búsquedas por título, así como la persistencia de los datos.

Referencias Bibliográficas

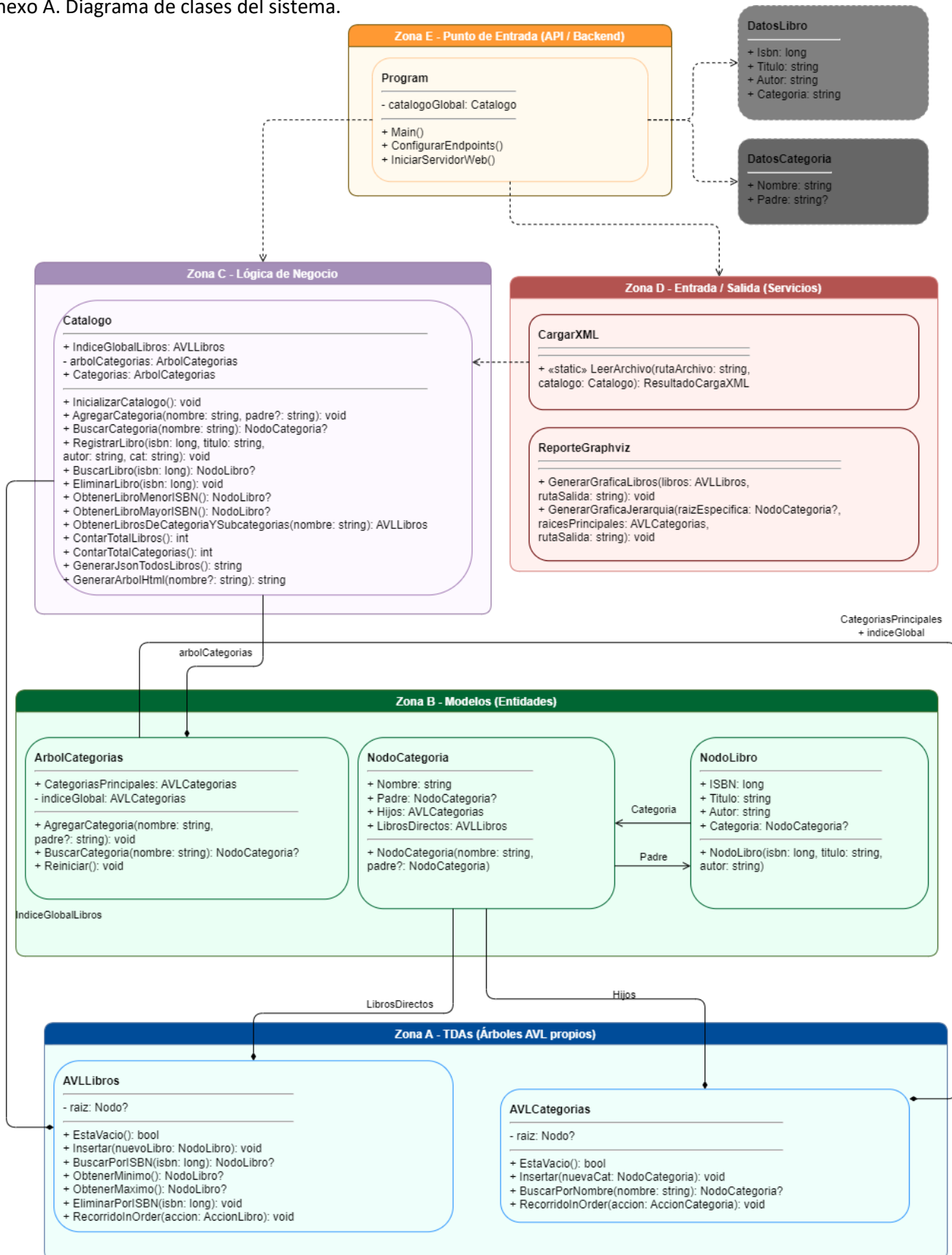
Auxiliatura de Introducción a la Programación y Computación 2. (2026). *Proyecto 2: Catalogo de Libros* [Página web]. <https://proyectos-auxiliatura-ipc2.vercel.app/proyecto2>

Facultad de Ingeniería, Universidad de San Carlos de Guatemala. (2026). *Proyecto 2: Catalogo de Libros* [Enunciado de proyecto]. Introducción a la Programación y Computación 2.

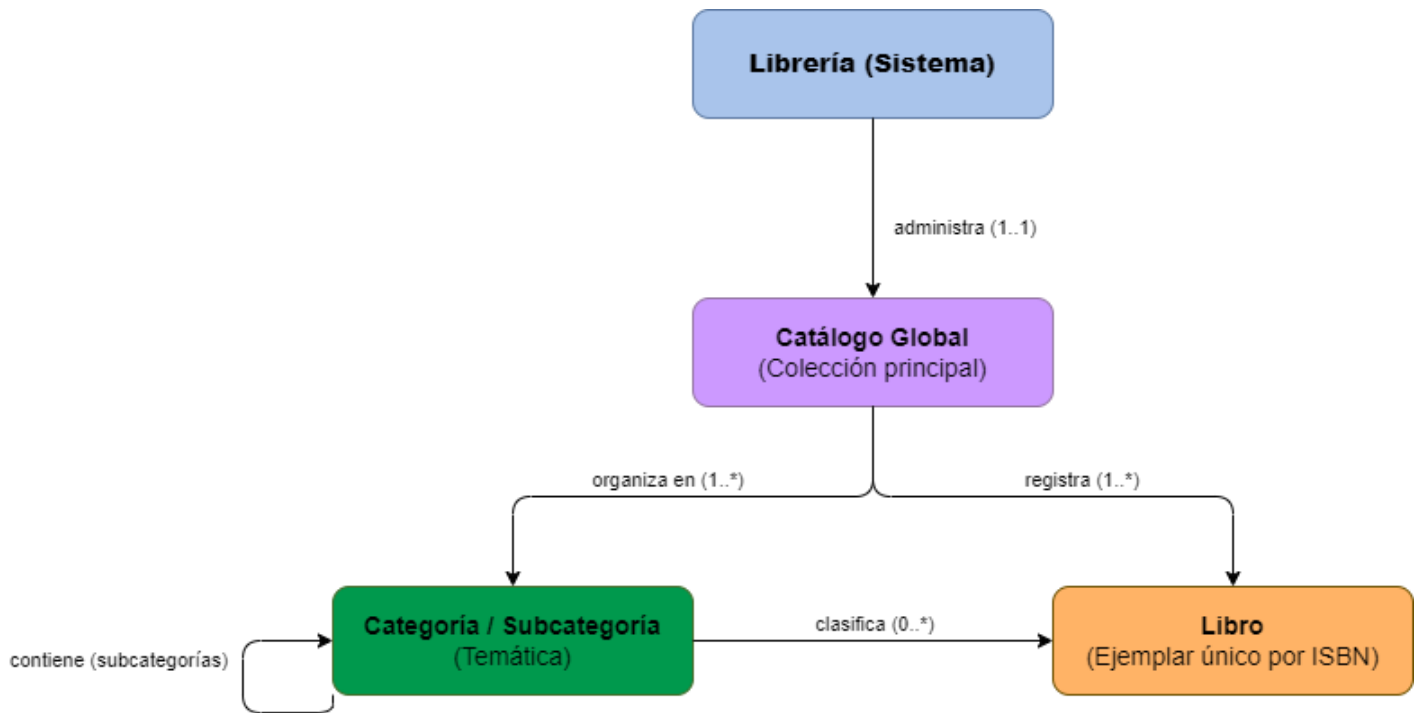
Gansner, E. R., & North, S. C. (2000). An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11), 1203–1233.

Anexos

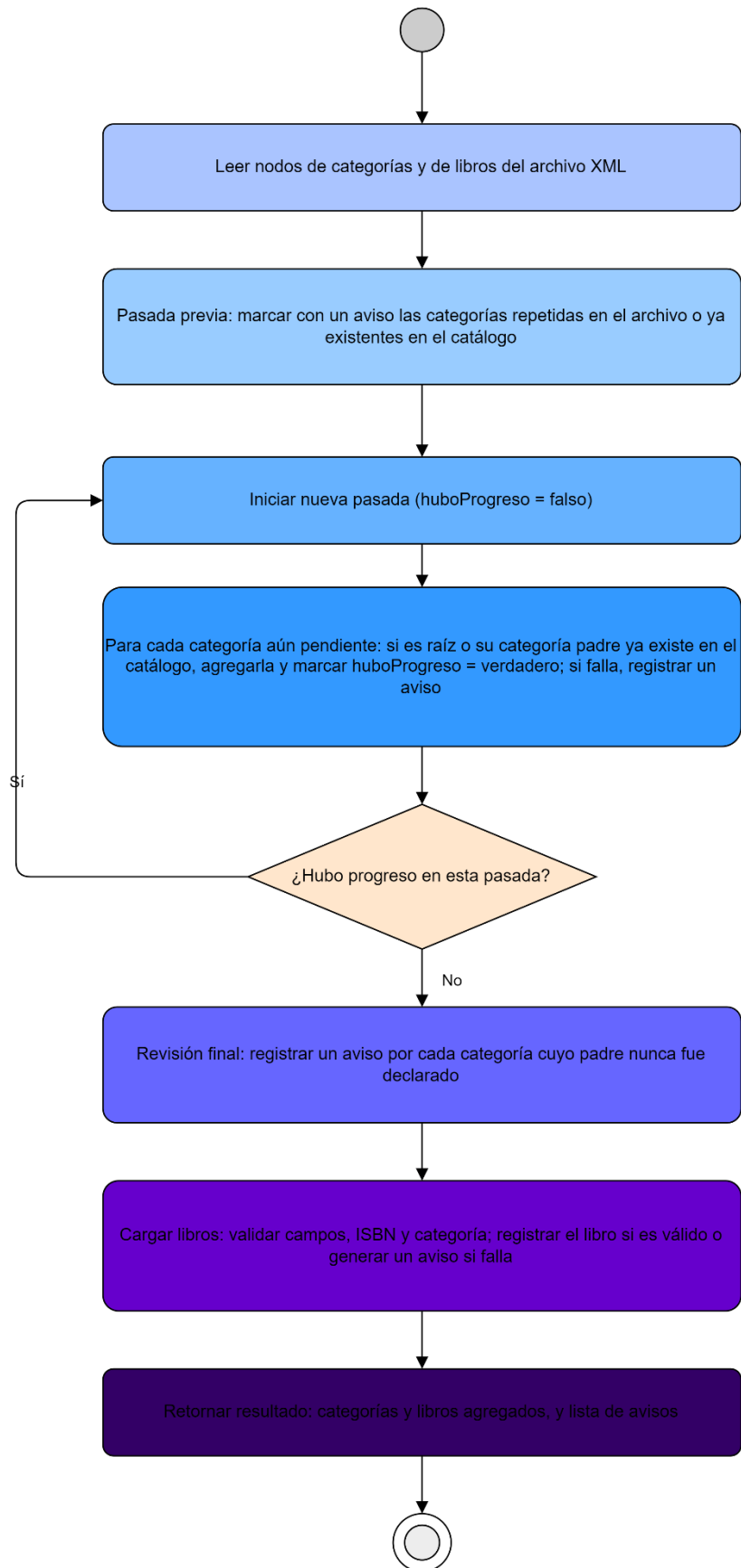
Anexo A. Diagrama de clases del sistema.



Anexo B. Modelo Conceptual



Anexo C. Actividades, Carga incremental de XML



Anexo D. Diagrama de Actividad – Búsqueda en árbol AVL

